



## Tipos de Estructura de Datos

Se clasifican en dos grupos

Estáticas VS Dinámicas

Lineales VS No Lineales



# Estáticas VS Dinámicas



# Estructuras Estáticas

- Las estructuras de datos hasta ahora vistas se almacenan estáticamente en la memoria física del ordenador.

El espacio de memoria se reserva con anticipación y no cambia durante la ejecución del programa

Esto permite una comprobación de tipos en tiempo de compilación.

- Inconvenientes de la configuración estática:

Su rigidez, ya que estas estructuras no pueden crecer durante la ejecución del programa.



Ejemplo:

Si trabajamos con un vector de 10 elementos

Ventajas

Desventajas





## Ventajas

Un array es una colección finita de datos del mismo tipo, que se almacenan en posiciones consecutivas de memoria y reciben un nombre común.

## Desventajas

Para ordenar datos físicamente hay que intercambiar dos datos de un lugar físico a otro de la estructura. La eliminación e inserción de datos también implica corrimiento de elementos.



# Estructura Dinámicas

Para saltar esta rigidez se trabaja con estructuras de datos dinámicas que al contrario que las estructuras estáticas; no tiene ningún tipo de limitación permitiendo que el tamaño variable de una estructura cuyos valores **crecen o decrecen conforme se ejecuta el programa.**





# Lineales VS No Lineales



# **Estructuras de Datos lineales**





## Estructuras de Datos lineales

Las estructuras lineales de datos se caracterizan porque sus elementos están en secuencia, relacionados en forma lineal, uno luego del otro.

Cada elemento de la estructura puede estar conformado por uno o varios subelementos o campos.





- Estructuras de datos lineales:

- Listas

- Pilas

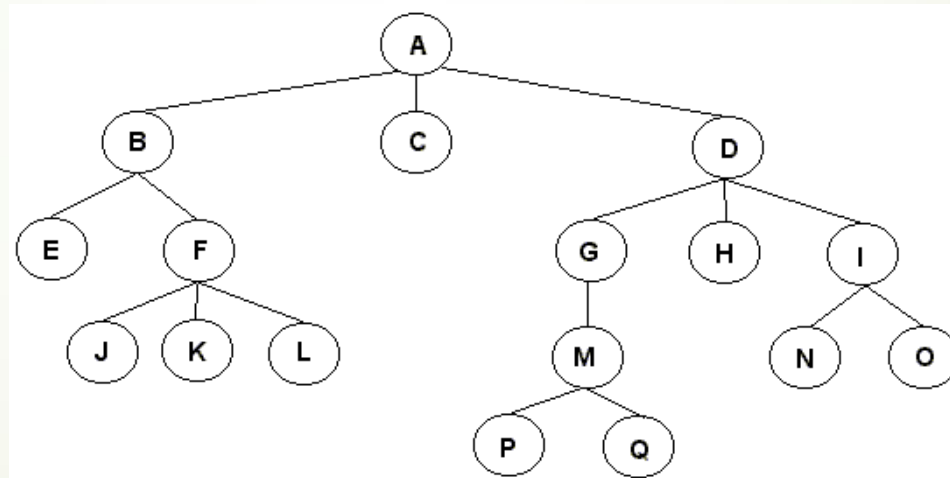
- Colas



# **Estructuras de Datos NO lineales**

## Por ejemplo Arboles

Son estructura que permiten representar datos que tienen enlaces jerárquicos entre ellos.





Para empezar a trabajar con estructura  
dinámicas  
tenemos que ver punteros

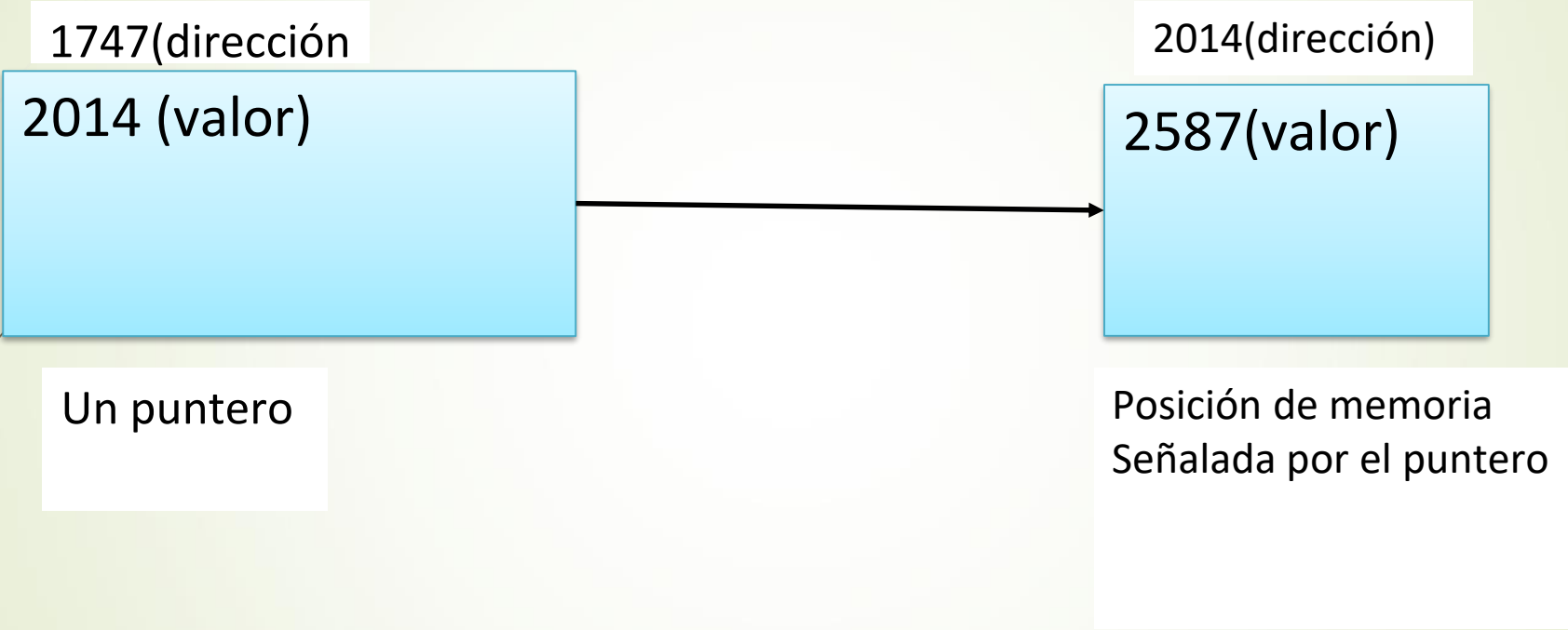


# **PUNTEROS**



## PUNTE ROS

- Un puntero es una variable que contiene una dirección de memoria.
- Por lo general, una variable contiene un valor y un puntero a ella contiene la dirección de dicha variable.
- Es decir que la variable se refiere **directamente** a un valor mientras que el puntero lo hace **indirectamente**.



1747(dirección

The diagram illustrates a pointer variable. On the left, a light blue box represents the pointer's memory location. Above it, a white box contains the address '1747(dirección'. Inside the blue box, the value '2014 (valor)' is displayed. Below the blue box, a white box contains the text 'Un puntero'. A horizontal arrow points from the right side of the blue box to the left side of another light blue box on the right. Above this second box, a white box contains the address '2014(dirección'. Inside the second blue box, the value '2587(valor)' is displayed. Below the second blue box, a white box contains the text 'Posición de memoria Señalada por el puntero'. A red arrow points from the left edge of the image towards the first blue box. Several curved lines are visible on the left side of the image.

2014 (valor)

Un puntero

2014(dirección)

2587(valor)

Posición de memoria  
Señalada por el puntero





Un puntero es una variable que se utiliza entonces para almacenar la dirección de otra variable, que es la que va a contener el dato a almacenar.

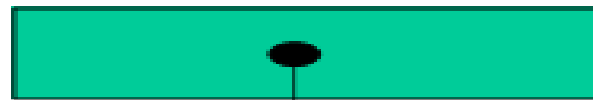
Es decir que cuando se trabaja con un puntero, en realidad hay dos variables implicadas:

- **la variable puntero** (quien contiene la dirección de memoria de la variable de datos)
- **la variable apuntada** (a quien apunta el puntero y quien contiene el dato en sí mismo)

## REPRESENTACIÓN GRÁFICA

Puntero

Variable  
referenciada





# Punteros

- Ventaja :

La flexibilidad que poseen las estructuras dinámicas en cuanto a las formas que pueden adoptar: árboles, listas, redes, etc..

- Inconvenientes:

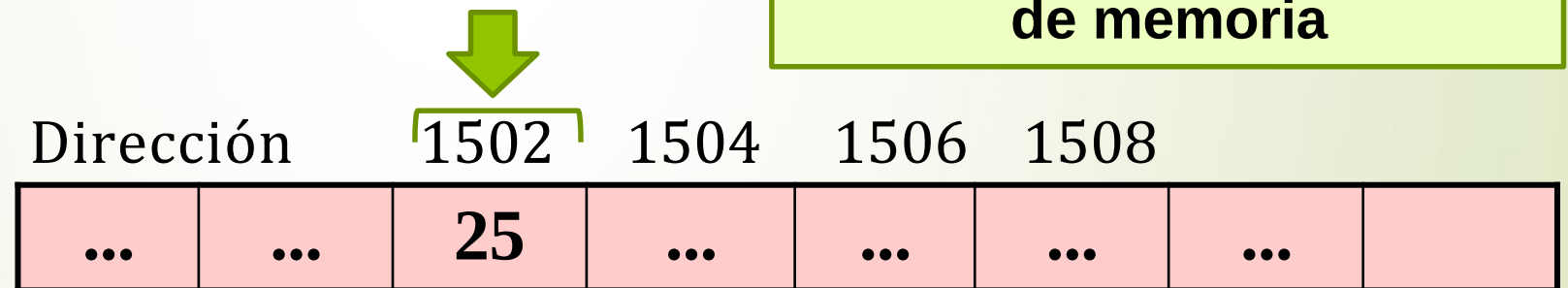
Gestión de la memoria: Su uso requiere una especial atención de la memoria disponible así como de la que ya no queramos utilizar.

## DIRECCIÓN Y CONTENIDO DE MEMORIA

- Una dirección de memoria y su contenido no es lo mismo.

```
int x = 25;
```

Un **puntero** es una variable que contiene una **dirección de memoria**



La **dirección** de la variable `x` es 1502

El **contenido** de la variable `x` es 25



# **PUNTEROS EN C**

# DECLARACIÓN DE PUNTEROS

## ○ Ejemplo

```
int *countPtr, count;
```



Puntero a un  
entero



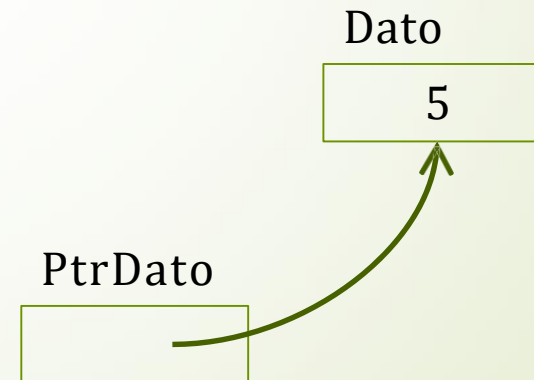
Es un entero  
(NO un puntero)

- El \* no se aplica a todos los nombres de variables de una declaración. Cada puntero debe llevar su nombre precedido por \*.

## OPERADORES DE PUNTEROS

- El operador & u *operador de dirección*, es un operador unario que retorna la dirección de su operando.
- Ejemplo**

```
int Dato = 5;  
int *PtrDato;  
PtrDato = &Dato;
```



## OPERADORES DE PUNTEROS

- El operador \*, también llamado *operador de indirección*, retorna el valor del objeto hacia el cual apunta su operando.

- Ejemplo**

```
int Dato = 5, *PtrDato;  
  
PtrDato = &Dato;  
  
printf("%d\n", *PtrDato);
```



Imprime 5



# PUNTEROS

- El puntero debe contener una dirección a un elemento del mismo tipo que la variable apuntada.

```
#include <stdio.h>
int main()
{   int *ptr;
    int dato=30;

    ptr = &dato;
    *ptr = 50;

    printf("Dato = %d\n", dato);

    return 0;
}
```

Declara un puntero a un entero

# PUNTEROS

- El puntero debe contener una dirección a un elemento del mismo tipo que la variable apuntada.

```
#include <stdio.h>
int main()
{   int *ptr;
    int dato=30;

    ptr = &dato;
    *ptr = 50;

    printf("Dato = %d\n", dato);

    return 0;
}
```

**&** es el **operador de dirección**: permite obtener la dirección de memoria de la variable que le sigue

# PUNTEROS

- El puntero debe contener una dirección a un elemento del mismo tipo que la variable apuntada.

```
#include <stdio.h>
int main()
{   int *ptr;
    int dato=30;

    ptr = &dato;
    *ptr = 50;

    printf("Dato = %d\n", dato);

    return 0;
}
```

No hay que confundir el \*  
que aparece en la  
**declaración**  
con  
el **operador de  
indirección**

# PUNTEROS

- El puntero debe contener una dirección a un elemento del mismo tipo que la variable apuntada.

```
#include <stdio.h>
int main()
{   int *ptr;
    int dato=30;

    ptr = &dato;
    *ptr = 50;

    printf("Dato = %d\n", dato);

    return 0;
}
```



Cámbielo por  
**float \* ptr**

Ejecute y observe el  
resultado obtenido

## QUÉ IMPRIME?

```
#include <stdio.h>
int main(void) {
    int a, b, c, *p1, *p2;

    p1 = &a;
    *p1 = 1;
    p2 = &b;
    *p2 = 2;
    p1 = p2;
    *p1 = 0;
    p2 = &c;
    *p2 = 3;
    printf("%d %d %d\n", a, b, c);

    return 0;
}
```

Imprime

**1 0 3**

## VISUALIZANDO EL VALOR DE UN PUNTERO

- Puede utilizarse printf con la especificación de conversión **%p** para visualizar el valor de una variable puntero en forma de entero hexadecimal.
- **Ejemplo**

```
int Dato = 5, *PtrDato;  
PtrDato = &Dato;  
printf("%p\n", PtrDato);
```

Qué imprime?



## VISUALIZANDO EL VALOR DE UN PUNTERO

- Puede utilizarse printf con la especificación de conversión **%p** para visualizar el valor de una variable puntero en forma de entero hexadecimal.
- **Ejemplo**

```
int Dato = 5, *PtrDato;  
PtrDato = &Dato;  
printf("%p\n", PtrDato);
```

← 0028FF1C



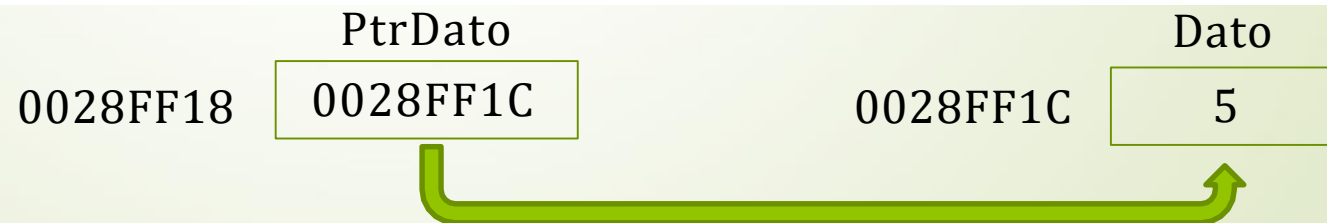
```
/* Operadores & y * */
#include <stdio.h>
int main()
{   int Dato = 5, *PtrDato;
    PtrDato = &Dato;
    printf("\n La direccion de Dato es %p\n"
           " El valor de PtrDato es %p\n\n",
           &Dato, PtrDato);

    printf(" La valor de Dato es %d\n"
           " El valor de *PtrDato es %d\n\n",
           Dato, *PtrDato);

    printf(" Note la relacion entre * y & \n"
           " &*PtrDato = %p\n *&PtrDato = %p\n",
           &*PtrDato, *&PtrDato);

    return 0;
}
```

Que imprime?





## SALIDA DEL PROGRAMA ANTERIOR

```
La direccion de Dato es 0028FF1C  
El valor de PtrDato es 0028FF1C
```

```
La valor de Dato es 5  
El valor de *PtrDato es 5
```

```
Note la relacion entre * y &  
&*PtrDato = 0028FF1C  
*&PtrDato = 0028FF1C
```





## PUNTE ROS

- Las **direcciones de memoria** dependen de la arquitectura de la computadora y de la gestión que el sistema operativo haga de ella.
- Desde C no es posible indicar numéricamente una dirección de memoria para guardar información (esto se hace a través de funciones específicas).
- Utilizamos punteros para acceder a la información a través de su dirección de memoria.



## INICIALIZACIÓN DE PUNTEROS

- Los punteros deben ser inicializados.
- Utilice el identificador **NULL** (definido en <stdio.h>) para indicar que el puntero no apunta a nada.
- El **0** es el único valor entero que puede asignarse directamente a un puntero y es equivalente a **NULL**.
- Cuando se asigna **0** a un puntero se realiza un casting previo automático al tipo apropiado.

## EJEMPLO

```
#include <stdio.h>
int main()
{
    int *ptr1 = 45637325;

    int *ptr2 = 0;

    int *ptr3 = NULL;

    return 0;
}
```



No es posible asignarle un valor fijo a un puntero. No es posible saber si es una posición válida.

## EJEMPLO

```
#include <stdio.h>
int main()
{
    int *ptr1 = 45637325;

    int *ptr2 = 0;      ←
    int *ptr3 = NULL;

    return 0;
}
```

El 0 es el único valor que puede asignarse a un puntero.  
La conversión a (int \*) es automática.

## EJEMPLO

```
#include <stdio.h>
int main()
{
    int *ptr1 = 45637325;

    int *ptr2 = 0;

    int *ptr3 = NULL;

    return 0;
}
```

NULL equivale a 0 y está  
definido en <stdio.h>



## PUNTEROS (resumen)

- Permiten simular el pasaje de parámetros por referencia.
  - Permiten crear y manipular estructuras de datos dinámicas.
  - Su manejo es de fundamental importancia para programar en C.
- 



# **PASAJE DE PARÁMETROS POR REFERENCIA**





## PASAJE DE PARÁMETROS POR REFERENCIA

- Vimos que en C los parámetros de las funciones siempre se pasan por valor.
  - Para simular el pasaje de parámetro por referencia se utiliza la dirección de la variable, es decir, que lo que se envía es un puntero a su valor.
  - El puntero es un parámetro sólo de entrada que permite modificar el valor de la variable a la que apunta.
- 

```
/* parámetro por referencia */
#include <stdio.h>
void cuadrado(int *);
int main()
{   int a = 5;

    printf("Valor original = %d\n", a);

    cuadrado(&a); ← Envía la dirección de la variable

    printf("Valor al cuadrado = %d\n", a);

    return 0; ← Recibe un puntero a un entero
}
void cuadrado(int * nro)
{
    *nro = *nro * *nro; ← Valor de la variable
                        apuntada por nro
}
```

## ¿QUÉ IMPRIME?

```
#include <stdio.h>

void duplicar( int nro );
int main()
{
    int nro = 3;
    printf("nro = %d\n", nro);
    duplicar(nro);
    printf("nro retornado =  %d\n", nro);
    return 0;
}

void duplicar( int nro ){
    nro = 2 * nro;
    printf("nro dentro de la funcion = %d\n", nro);
}
```